

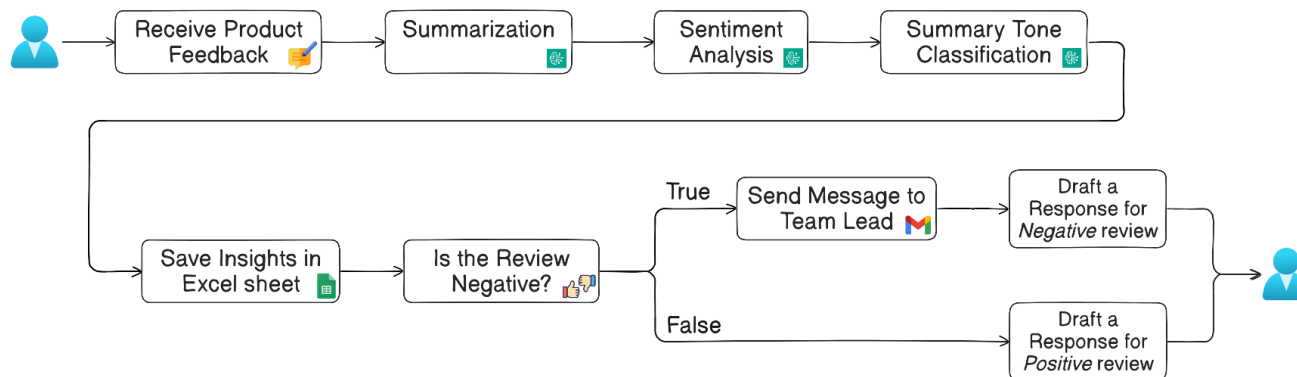
Advanced Certification Programme in Agentic and Generative AI

A Programme by IISc and TalentSprint

Assignment Notebook: **AI-Powered Customer Review Insights Pipeline**

Objective

Build an Automation pipeline that processes customer reviews for the *XSound Pro Headphones* (or any other product), then performs summarization, sentiment analysis, summary tone classification, exports the results to Excel sheet for ongoing trend monitoring and internal product feedback analysis, and if the review is negative send the insights to the team lead.



Problem Statement

For an online consumer electronics brand selling headphones, monitoring customer feedback is crucial for improving product quality, marketing strategies, and customer satisfaction.

Manually reviewing hundreds of customer reviews is **time-consuming**, often **inconsistent**, and prone to human bias.

Key challenges include:

- Identifying **core feedback themes** hidden in lengthy or subjective reviews.
- Evaluating **overall sentiment trends** for quick decision-making.
- Providing **actionable summaries** to marketing, product, and support teams in a timely manner.

An **automated AI-powered review analysis pipeline** can solve these challenges by:

- Reducing processing time from hours to minutes.

- Ensuring consistent sentiment and tone detection across all reviews.
- Delivering structured summaries and sentiment scores directly to a centralized platform like Google Sheets.

This experiment should solve the above problem by using **LangChain** to build a Generative AI application that leverages LLMs for generating Product Review Summaries from user-entered Product Name and Product Review.

Use Case Scenario

The sales team of an organization needs a streamlined way to analyze customer product reviews and quickly understand key insights without manual processing. Currently, generating concise review summaries, identifying sentiment, and detecting tone are manual and time-consuming tasks. This slows down follow-ups with customers and limits timely feedback to internal stakeholders.

To address this, you will need to implement a LangChain pipeline that should be able to:

- Accept user-entered **Product Name** and **Product Review**
- Generate a **Product Review Summary** using an LLM like from OpenAI models, or Mistral Cloud Chat models
- Extract both the **Sentiment** of the review and the **Tone** of the generated summary
- Calculate confidence scores for sentiment and tone
- Log all results (including scores) into an Excel Sheet for internal sales team's reference
- Automatically send email message if the customer review is negative with the product name and AI-generated summary

✓ Setup Steps:

> Please enter your registration id to start:

Id

2513936

[Show code](#)

> Please enter your password (your registered phone number) to continue:

password

8309823730

[Show code](#)

> Run this cell to complete the setup for this Notebook

[Show code](#)

Setup completed successfully

Install Required Libraries

```
%%capture
!pip -q install groq==0.37.1
!pip -q install langchain-groq==1.1.2
!pip -q install langchain==1.2.12
!pip -q install langchain-core==1.2.19
!pip -q install langchain_community==0.4.1
```

Read the Groq API Key

Using the Groq API Key, you will have access to the OpenAI and Llama model, free of cost, under the [Free-tier](#).

This cell retrieves the Groq API key securely stored in Google Colab's Secrets (see 'key' icon on the leftmost panel of this notebook) and sets it as an environment variable `GROQ_API_KEY`. This is a recommended practice to avoid exposing your API key directly in the code.

- Go to <https://console.groq.com/>, and setup a Free account.
- Create a new API by visiting: <https://console.groq.com/keys>
- Save the key in Google Colab's Secrets

```
Secret Name: GROQ_API_KEY
Secret Value: Paste your Groq api key
```

- Read the key and save as an environment variable `GROQ_API_KEY`

```
# Save the key in Colab's Secrets then load from there

import os
from google.colab import userdata

os.environ['GROQ_API_KEY'] = userdata.get('GROQ_API_KEY')
```

Initialize a Chat Model

```
from langchain_groq import ChatGroq

llm = ChatGroq(
    model="openai/gpt-oss-20b",
    temperature=0,
)
```

```
response = llm.invoke("What is 2 plus 3")

print(response.content)
```

```
2 + 3 = 5.
```

Summarization

Perform summarization for the Product Review submitted. (Output should only be the summary text)

```
summarization_prompt = """
You are an expert product analyst.

Your task is to read the customer product review and generate a concise, clear summary.

Instructions:
- Capture the key points of the review.
- Include both positive and negative aspects if present.
- Keep the summary short (2-3 sentences maximum).
- Do not add any new information.
- Do not include explanations, labels, or formatting.

Output:
Return only the summary text.

Product Name: {product_name}
Product Review: {product_review}
"""
```

```
from langchain_core.prompts import ChatPromptTemplate
```

```
summarization_template = ChatPromptTemplate.from_template(summarization_prompt)
summarization_template
```

```
ChatPromptTemplate(input_variables=['product_name', 'product_review'], input_types={}, partial_variables={}, messages=[HumanMessagePromptTemplate(prompt=PromptTemplate(input_variables=['product_name', 'product_review'], input_types={}, partial_variables={}, template='\nYou are an expert product analyst.\n\nYour task is to read the customer product review and generate a concise, clear summary.\n\nInstructions:\n- Capture the key points of the review.\n- Include both positive and negative aspects if present.\n- Keep the summary short (2-3 sentences maximum).\n- Do not add any new information.\n- Do not include explanations, labels, or formatting.\n\nOutput:\nReturn only the summary text.\n\nProduct Name: {product_name}\nProduct Review: {product_review}\n'), additional_kwargs={})])
```

```
review1 = """
I've been using these Headphones for about two weeks now, and overall I'm quite impressed.
The sound quality is excellent with deep bass and clear vocals, making it great for both music and calls.
The noise cancellation works well in moderately noisy environments, though it struggles a bit in very loud places.
The battery life is solid and easily lasts a full day of use.
However, the ear cushions start to feel slightly uncomfortable after long hours, and
the build quality could be a bit more premium for the price.
Still, it's a good value for money option.
"""
```

```
print(summarization_template.invoke(
    {"product_name": "XSound Pro Headphones",
     "product_review": review1}
    ).messages[0].content)
```

You are an expert product analyst.

Your task is to read the customer product review and generate a concise, clear summary.

Instructions:

- Capture the key points of the review.
- Include both positive and negative aspects if present.
- Keep the summary short (2-3 sentences maximum).
- Do not add any new information.
- Do not include explanations, labels, or formatting.

Output:

Return only the summary text.

Product Name: XSound Pro Headphones

Product Review:

I've been using these Headphones for about two weeks now, and overall I'm quite impressed.

The sound quality is excellent with deep bass and clear vocals, making it great for both music and calls.

The noise cancellation works well in moderately noisy environments, though it struggles a bit in very loud places.

The battery life is solid and easily lasts a full day of use.

However, the ear cushions start to feel slightly uncomfortable after long hours, and the build quality could be a bit more premium for the price.

Still, it's a good value for money option."

Initialize Output parser

```
from langchain_core.output_parsers import StrOutputParser
```

```
str_parser = StrOutputParser()
```

Create Summarization Chain

```
summarization_chain = summarization_template | llm | str_parser
```

Execute Chain

```
summary = summarization_chain.invoke(
    {"product_name": "XSound Pro Headphones",
     "product_review": review1}
)
```

```
summary
```

```
'The XSound Pro headphones deliver excellent sound with deep bass, clear vocals, and solid battery life, making them a good value for money. Noise cancellation works well in moderate noise but falters in very loud environments. However, the ear cushions become uncomfortable after long use and the build quality feels less premium than expected.'
```

▼ Sentiment Analysis

Analyse the sentiment (Positive/Negative/Neutral) of the Product Review submitted.

Output should be in this format:

```
Sentiment: <Positive/Negative/Neutral> | Score: <numeric value between 0 and 1>
```

```
sentiment_prompt = """
You are a sentiment analysis expert.
```

```
Your task is to analyze the sentiment of the given product review.
```

```
Instructions:
```

- Classify the sentiment as one of the following: Positive, Negative, or Neutral.
- Assign a confidence score between 0 and 1 (e.g., 0.85).
- Base your decision only on the content of the review.
- Do not include any explanation or additional text.

```
Output Format:
```

```
Sentiment: <Positive/Negative/Neutral> | Score: <numeric value between 0 and 1>
```

```
Product Review:
```

```
{product_review}
"""
```

```
from langchain_core.prompts import ChatPromptTemplate
```

```
sentiment_template = ChatPromptTemplate.from_template(sentiment_prompt)
sentiment_template
```

```
ChatPromptTemplate(input_variables=['product_review'], input_types={}, partial_variables={}, messages=[HumanMessagePromptTemplate(prompt=PromptTemplate(input_variables=['product_review'],
input_types={}, partial_variables={}, template='\nYou are a sentiment analysis expert.\n\nYour task is to analyze the sentiment of the given product review.\n\nInstructions:\n- Classify
the sentiment as one of the following: Positive, Negative, or Neutral.\n- Assign a confidence score between 0 and 1 (e.g., 0.85).\n- Base your decision only on the content of the
review.\n- Do not include any explanation or additional text.\n\nOutput Format:\nSentiment: <Positive/Negative/Neutral> | Score: <numeric value between 0 and 1>\n\nProduct
Review:\n{product_review}\n'), additional_kwargs={}]])
```

```
print(sentiment_template.invoke(
    {"product_review": review1}
).messages[0].content)
```

```
You are a sentiment analysis expert.
```

```
Your task is to analyze the sentiment of the given product review.
```

```
Instructions:
```

- Classify the sentiment as one of the following: Positive, Negative, or Neutral.
- Assign a confidence score between 0 and 1 (e.g., 0.85).
- Base your decision only on the content of the review.
- Do not include any explanation or additional text.

```
Output Format:
```

```
Sentiment: <Positive/Negative/Neutral> | Score: <numeric value between 0 and 1>
```

```
Product Review:
```

```
I've been using these Headphones for about two weeks now, and overall I'm quite impressed.
The sound quality is excellent with deep bass and clear vocals, making it great for both music and calls.
The noise cancellation works well in moderately noisy environments, though it struggles a bit in very loud places.
The battery life is solid and easily lasts a full day of use.
However, the ear cushions start to feel slightly uncomfortable after long hours, and
the build quality could be a bit more premium for the price.
Still, it's a good value for money option."
```

```
# Create Sentiment Analysis Chain

sentiment_chain = sentiment_template | llm | str_parser
```

```
# Execute Chain

sentiment = sentiment_chain.invoke(
    {"product_review": review1}
)

sentiment

'Sentiment: Positive | Score: 0.90'
```

Summary Tone Classification

Classify the tone (Informative/Persuasive/Friendly) of the Product Review Summary generated during Summarization step.

Output should be in this format:

```
Tone: <Informative/Persuasive/Friendly> | Score: <numeric value between 0 and 1>
```

```
tone_clf_prompt = """
You are an expert in text tone analysis.

Your task is to classify the tone of the given product review summary.

Instructions:
- Classify the tone as one of the following:
  Informative, Persuasive, or Friendly.
- Assign a confidence score between 0 and 1 (e.g., 0.82).
- Base your decision only on the given summary.
- Do not include any explanation or additional text.

Output Format:
Tone: <Informative/Persuasive/Friendly> | Score: <numeric value between 0 and 1>

Product Review Summary:
{review_summary}
"""
```

```
from langchain_core.prompts import ChatPromptTemplate
```

```
tone_clf_template = ChatPromptTemplate.from_template(tone_clf_prompt)
tone_clf_template
```

```
ChatPromptTemplate(input_variables=['review_summary'], input_types={}, partial_variables={}, messages=[HumanMessagePromptTemplate(prompt=PromptTemplate(input_variables=['review_summary'],
input_types={}, partial_variables={}, template='\nYou are an expert in text tone analysis.\n\nYour task is to classify the tone of the given product review summary.\n\nInstructions:\n-
Classify the tone as one of the following:\n  Informative, Persuasive, or Friendly.\n- Assign a confidence score between 0 and 1 (e.g., 0.82).\n- Base your decision only on the given
```

```
summary.\n- Do not include any explanation or additional text.\n\nOutput Format:\nTone: <Informative/Persuasive/Friendly> | Score: <numeric value between 0 and 1>\n\nProduct Review Summary:\n{n{review_summary}\n'}, additional_kwargs={})]]
```

```
print(tone_clf_template.invoke(
    {"review_summary": summary}
).messages[0].content)
```

You are an expert in text tone analysis.

Your task is to classify the tone of the given product review summary.

Instructions:

- Classify the tone as one of the following:
Informative, Persuasive, or Friendly.
- Assign a confidence score between 0 and 1 (e.g., 0.82).
- Base your decision only on the given summary.
- Do not include any explanation or additional text.

Output Format:

Tone: <Informative/Persuasive/Friendly> | Score: <numeric value between 0 and 1>

Product Review Summary:

The XSound Pro headphones deliver excellent sound with deep bass, clear vocals, and solid battery life, making them a good value for money. Noise cancellation works well in moderate noise

```
# Create Tone Classification Chain
```

```
tone_clf_chain = tone_clf_template | llm | str_parser
```

```
# Execute Chain
```

```
tone = tone_clf_chain.invoke(
    {"review_summary": summary}
)
```

```
tone
```

```
'Tone: Informative | Score: 0.92'
```

▼ Save Insights in the Excel Sheet

```
import pandas as pd
import os
```

```
FILENAME = "Customer_Review_Insights.xlsx"
```

```
product_name = "XSound Pro Headphones"
product_review = review1      # Using the review1 variable defined earlier
```

```
def save_to_excel(product_name, product_review, summary, sentiment, tone, filename=FILENAME):
    # Prepare the data for the new row
    new_row_data = {
```



```
"Product Name": [product_name],
"Product Review": [product_review],
"Product Review Summary": [summary],
"Sentiment and Score": [sentiment],
"Tone and Score": [tone]
}
new_df = pd.DataFrame(new_row_data)

# Check if the file exists
if os.path.exists(filename):
    # If it exists, load the existing data and append the new row
    existing_df = pd.read_excel(filename)
    updated_df = pd.concat([existing_df, new_df], ignore_index=True)
else:
    # If it doesn't exist, the new_df is the first DataFrame
    updated_df = new_df

# Save the DataFrame to Excel
updated_df.to_excel(filename, index=False)
print(f"Insights saved to `{filename}` successfully!")
```

```
# Execute the function to save insights
save_to_excel(
    product_name=product_name,
    product_review=product_review,
    summary=summary,
    sentiment=sentiment,
    tone=tone
)
```

Insights saved to `Customer\_Review\_Insights.xlsx` successfully!

› Run this cell to check the Data stored in the Excel Sheet

Show code

Customer_Review_Insights.xlsx

	Product Name	Product Review	Product Review Summary	Sentiment and Score	Tone and Score
0	XSound Pro Headphones	\nI've been using these Headphones for about two weeks now, and overall I'm quite impressed.\nThe sound quality is excellent with deep bass and clear vocals, making it great for both music and calls.\nThe noise cancellation works well in moderately noisy environments, though it struggles a bit in very loud places.\nThe battery life is solid and easily lasts a full day of use.\nHowever, the ear cushions start to feel slightly uncomfortable after long hours, and\nthe build quality could be a bit more premium	The XSound Pro headphones deliver excellent sound with deep bass, clear vocals, and solid battery life, making them a good value for money. Noise cancellation works well in moderate noise but falters in very loud environments. However, the ear cushions become uncomfortable after long use and the build quality feels less	Sentiment: Positive Score: 0.90	Tone: Informative Score: 0.92

↘ Send Email Message for Negative Reviews

- Add the condition to check if the sentiment of the Product review is Negative or not.
- If the review is Negative, send an email message to the Team Lead with following details:
 - **Send To:** <Provide your email>

- **Subject:** Negative Customer Review Received – <Product-Name>
- **Message:**

Dear Team Lead

We have received a negative customer review for the product - <Product-Name>.

Please find the feedback summary below:

<Product Review Summary>

Best Regards,
Automation Team

✓ How to Send Email using Python?

To send an email message programatically using Python, we need to use App Password.

How to create and use an App Password?

An app password is a 16-digit passcode that gives a less secure app or device permission to access your Google Account.

Note: App passwords can only be used with Google accounts that have 2-Step Verification turned on.

To get your account App Password,

1. Go to your Google Account.
2. Select Security.
3. In the search bar, search for App passwords.
4. Enter a name that helps you remember where you'll use the app password.
5. Select Generate.
6. Copy the generated app password. Remove spaces in between, it is a 16-character code.
7. Select Done.

If you've set up 2-Step Verification but can't find the option to add an app password, it might be because:

1. Your Google Account has 2-Step Verification set up only for security keys.
2. You're logged into a work, school, or another organization account.
3. Your Google Account has Advanced Protection.

```
# Standard library for sending email using the Simple Mail Transfer Protocol (SMTP)
import smtplib

# Classes for creating email content and multi-part (e.g., text + attachments) messages
from email.mime.text import MIMEText
from email.mime.multipart import MIMEMultipart

# Utility to format the "From" header with a display name plus email address
from email.utils import formataddr
```

```

SENDER_EMAIL = "singhaditya218@gmail.com"    # <----- REPLACE with your own email address
RECEIVER_EMAIL = "singhaditya218@gmail.com"   # <----- REPLACE with your own email address
APP_PASSWORD = "ssaaii22"                    # <----- REPLACE with your app password

```

```
# Function to send Email
```

```

def send_email(receiver_email, subject, body):
    """
    Send a plain-text email using Gmail's SMTP server.

    Parameters:
        receiver_email (str): Recipient's email address
        subject (str): Subject line of the email
        body (str): Message body (plain text)
    """

    # ---- SMTP Configuration ----
    smtp_server = "smtp.gmail.com"           # Gmail SMTP server
    smtp_port = 587                          # Port for TLS encryption
    sender_name = "Support Team"             # Display name that appears to the recipient
    sender_email = SENDER_EMAIL
    app_password = APP_PASSWORD

    # ---- Compose the Email ----
    message = MIMEMultipart()                # Create a multi-part email container
    message["From"] = formataddr((sender_name, sender_email)) # Add sender name + email
    message["To"] = receiver_email           # Add recipient address
    message["Subject"] = subject             # Add subject line
    message.attach(MIMEText(body, "plain"))  # Attach the email body as plain text

    # ---- Send the Email ----
    try:
        # Connect to the SMTP server and secure the connection
        with smtplib.SMTP(smtp_server, smtp_port) as server:
            server.starttls()                 # Start TLS encryption for security
            server.login(sender_email, app_password) # Log in using the sender's credentials
            server.sendmail(sender_email, receiver_email, message.as_string()) # Send the email
        return "Email sent successfully!"
    except Exception as e:
        # Print and return an error message if sending fails
        print(f"Error sending message: {e}")
        return "Email could not be delivered!"

```

```
# Test the function
```

```

send_email(
    receiver_email=RECEIVER_EMAIL,
    subject = "Test Email",
    body = "This is a test email sent using Python."
)

```

```

Error sending message: (535, b'5.7.8 Username and Password not accepted. For more information, go to\n5.7.8 https://support.google.com/mail/?p=BadCredentials 6a1803df08f44-8a593908b47sm16
'Email could not be delivered!')

```

```
# Function to Send Alert Notification

def send_alert_notification(product_name, product_review, summary, sentiment, tone):
    """
    Send an alert notification if the sentiment of the product review is negative.
    """

    # Check if the sentiment is negative
    if "Negative" in sentiment:
        send_email(
            receiver_email=RECEIVER_EMAIL,
            subject=f"Negative Customer Review Received - {product_name}",
            body=f"""
            Dear Team Lead

            We have received a negative customer review for the product - {product_name}.
            Please find the feedback summary below:
            {summary}

            Best Regards,
            Automation Team
            """
        )
        print("Received a Negative review. Alert notification sent!")
    else:
        print("Received a Positive/Neutral review. No alert notification sent.")
```

```
# Test the function
send_alert_notification(
    product_name=product_name,
    product_review=product_review,
    summary=summary,
    sentiment=sentiment,
    tone=tone
)
```

```
Received a Positive/Neutral review. No alert notification sent.
```

▼ Draft Responses for Positive and Negative Reviews

```
# Define a Jinja Template to draft response

from jinja2 import Template

template_str = """Thank you for taking the time to share your feedback regarding `{{ product_name }}`.

{% if sentiment == "negative" %}
We sincerely apologize for your experience and understand your concerns. Your feedback is extremely valuable to us, and we are actively working to address the issues you've highlighted.
Our team would like to investigate this further and ensure a quick resolution.

{% elif sentiment == "positive" %}
We are delighted to hear that you had a great experience and that the product met your expectations.
```

```
Your support and appreciation motivate us to continue delivering high-quality products and services.
```

```
{% else %}  
We appreciate your feedback.
```

```
{% endif %}  
"""
```

```
response_template = Template(template_str)
```

```
# Render the Template for Negative review  
output = response_template.render(product_name="XSound Pro Headphones", sentiment="negative")  
print(output)
```

Thank you for taking the time to share your feedback regarding `XSound Pro Headphones`.

We sincerely apologize for your experience and understand your concerns. Your feedback is extremely valuable to us, and we are actively working to address the issues you've highlighted. Our team would like to investigate this further and ensure a quick resolution.

```
# Render the Template for Positive review  
output = response_template.render(product_name="XSound Pro Headphones", sentiment="positive")  
print(output)
```

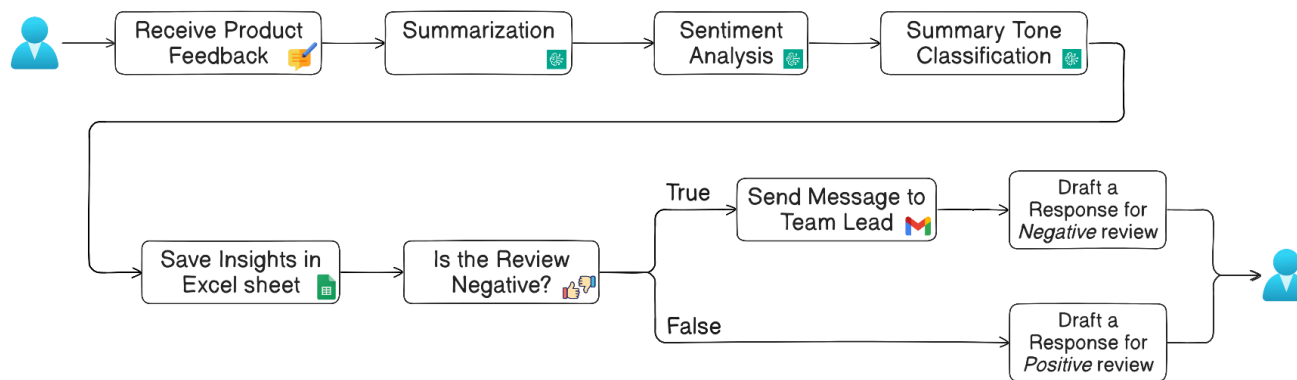
Thank you for taking the time to share your feedback regarding `XSound Pro Headphones`.

We are delighted to hear that you had a great experience and that the product met your expectations. Your support and appreciation motivate us to continue delivering high-quality products and services.

▼ Complete Pipeline

All individual components—feedback input, summarization, sentiment analysis, tone classification, saving the insights, alerting and responding—are integrated into a single **Gradio interface**.

AI-Powered Customer Review Insights Pipeline:



Test the pipeline on the sample reviews given below for *XSound Pro Headphones*:

- *"I've been using the XSound Pro Headphones for a few weeks now, and they have exceeded my expectations. The sound quality is crystal clear, with deep bass and well-balanced audio. The battery life easily lasts more than a full day of use. They are also very comfortable to wear for long periods. Overall, I think they offer great value for the price."*
- *"I'm quite disappointed with the XSound Pro Headphones after using them for a few days. The noise cancellation barely works and doesn't block out background sounds effectively. The audio quality feels flat compared to other brands in the same price range. The build also feels a bit cheap and not very durable. Overall, I don't think they are worth the money."*
- *"I've been using these headphones casually, and they seem decent for everyday use. The sound quality is acceptable but not particularly impressive. Battery life is average and gets me through most of the day. The design is simple and functional without any standout features. Overall, it's neither great nor bad."*
- *"The sound quality of the XSound Pro is really good, especially for music and videos. I like the clarity and bass performance. However, the ear cushions become uncomfortable after using them for more than an hour. Also, the charging time feels longer than expected. So while the audio is great, comfort could be improved."*

```

import gradio as gr

# Input elements for Gradio interface
in_product_name = gr.Textbox(label="Product Name", lines=2)
in_product_review = gr.Textbox(label="Product Review", lines=5)

# Output elements for Gradio interface
out_message = gr.Textbox(label="Message", lines=5)

# Function to get Review Insights
def get_review_insights(product_name, product_review):
    # Summarize the review
    summary = summarization_chain.invoke(
        {"product_name": product_name, "product_review": product_review}
    )
    # Sentiment of the review

```

```

sentiment = sentiment_chain.invoke(
    {"product_review": product_review}
)
# Tone of the review
tone = tone_clf_chain.invoke(
    {"review_summary": summary}
)
# Save the insights in the Excel sheet
save_to_excel(
    product_name=product_name,
    product_review=product_review,
    summary=summary,
    sentiment=sentiment,
    tone=tone
)
# If the sentiment is negative, send an email to Team Lead
if "Negative" in sentiment:
    send_alert_notification(
        product_name=product_name,
        product_review=product_review,
        summary=summary,
        sentiment=sentiment,
        tone=tone
    )
    return response_template.render(product_name=product_name, sentiment="negative")

return response_template.render(product_name=product_name, sentiment="positive")

```

Sample reviews to test the pipeline

```

sample_review1 = "I've been using the XSound Pro Headphones for a few weeks now, and they have exceeded my expectations. The sound quality is crystal clear, with deep bass and well-balanced treble. The noise cancellation barely works and doesn't block out background sounds effectively. Battery life is average."
sample_review2 = "I'm quite disappointed with the XSound Pro Headphones after using them for a few days. The noise cancellation barely works and doesn't block out background sounds effectively. Battery life is average."
sample_review3 = "I've been using these headphones casually, and they seem decent for everyday use. The sound quality is acceptable but not particularly impressive. Battery life is average."
sample_review4 = "The sound quality of the XSound Pro is really good, especially for music and videos. I like the clarity and bass performance. However, the ear cushions become uncomfortable after long sessions."

```

```

# Create Gradio Interface
interface = gr.Interface(
    fn=get_review_insights,
    inputs=[in_product_name, in_product_review],
    outputs=out_message,
    title="Customer Review Insights",
    description="Enter product name and review to get insights.",
    examples=[
        ["XSound Pro Headphones", sample_review1],
        ["XSound Pro Headphones", sample_review2],
        ["XSound Pro Headphones", sample_review3],
        ["XSound Pro Headphones", sample_review4]
    ],
    flagging_mode="never",
)

```

```

# Launch the Gradio Interface
interface.launch(debug=True)      # <-- for debugging purpose keep debug=True, keep it False

```

```
# NOTE that in the debugging mode, this code cell will keep on running.
```

It looks like you are running Gradio on a hosted Jupyter notebook, which requires `share=True`. Automatically setting `share=True` (you can turn this off by setting `share=False` in `launch`).

Colab notebook detected. This cell will run indefinitely so that you can see errors and logs. To turn off, set debug=False in launch().

* Running on public URL: <https://ea21eccec5e4e76525a.gradio.live>

This share link expires in 1 week. For free permanent hosting and GPU upgrades, run `gradio deploy` from the terminal in the working directory to deploy to Hugging Face Spaces (<https://huggingface.co/spaces/gradio/launch>)



No interface is running right now

```
Insights saved to `Customer_Review_Insights.xlsx` successfully!  
Keyboard interruption in main thread... closing server.  
Killing tunnel 127.0.0.1:7860 <> https://ea21eccec5e4e76525a.gradio.live
```

✓ Run this cell to check the Data stored in the Excel Sheet

```
##title Run this cell to check the Data stored in the Excel Sheet  
  
import pandas as pd  
  
pd.set_option('display.max_colwidth', None)  
  
# Check if the file exists  
if os.path.exists(FILENAME):  
    # If it exists, load the existing data and append the new row  
    df = pd.read_excel(FILENAME)  
else:  
    df = "File doesn't exist!"
```



```
print(FILENAME)
df
```

[Show hidden output](#)

✓ Please answer the questions below to complete the experiment:

› What is the primary purpose of prompt template in LangChain?

Answer

To dynamically generate structured prompts using variables

[Show code](#)

› How was the experiment?

Complexity

Good and Challenging for me

[Show code](#)

› If it was too easy, what more would you have liked to be added? If it was very difficult, what would you have liked to have been removed?

Additional

It was okay. Good for beginners

[Show code](#)

› Can you identify the concepts from the lecture which this experiment covered?

Concepts

Yes

[Show code](#)

› Text and image description/explanation and code comments within the experiment:

Comments

Very Useful

[Show code](#)

> Mentor Support:

Mentor_support

Very Useful

[Show code](#)

v Run this cell to submit your notebook for grading

```
##title Run this cell to submit your notebook for grading { vertical-output: true }
try:
    if submission_id:
        return_id = submit_notebook()
        if return_id : submission_id = return_id
    else:
        print("Please complete the setup first.")
except NameError:
    print ("Please complete the setup first.")
```

[Hide code](#)

Your submission is successful.
Ref Id: 985
Date of submission: 05 Apr 2026
Time of submission: 13:03:47
View your submissions: https://learn-iisc.talentsprint.com/notebook_submissions